

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report, cohen_kappa_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import roc_curve, roc_auc_score, RocCurveDisplay
```

```
#Normalizar apenas com os dados de treino para não ocorrer vazamento de dados e colocado "FIT" para os dados de test pelo m
scaler = StandardScaler()
scaler.fit(x_train_corr)
x_train_escalonado_corr = scaler.transform(x_train_corr) # X_TRAIN → Aprende + Transforma. O scaler "estuda" a média e desv
x_test_escalonado_corr = scaler.transform(x_test_corr) # X_TEST → Só Transforma. O scaler usa os parâmetros que APRENDEU de
```

8. MODELO DE MACHINE LEARNING

8.2 SVM

8.2.1 Criando Modelo com 11 variáveis

```
#Melhorando Hiperparametro
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
param_grid_svm = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 0.001, 0.01, 0.1, 1],
    'kernel': ['rbf']
}

grid_svm = GridSearchCV(SVC(random_state=42), param_grid_svm,
                        scoring='accuracy', cv=5, n_jobs=-1, verbose=1)
grid_svm.fit(x_train_escalonado, y_train)
print('SVM:', grid_svm.best_params_)
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits
SVM: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}

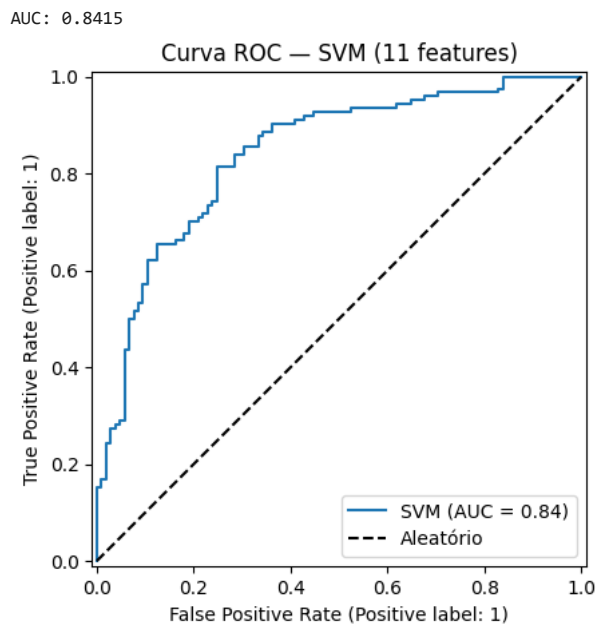
```
#resultado
svm = grid_svm.best_estimator_
y_pred_svm = svm.predict(x_test_escalonado)
print(f"Acurácia: {accuracy_score(y_test, y_pred_svm):.4f}")
```

Acurácia: 0.7860

8.2.2 Curva ROC - 11 variáveis

```
# Curva ROC
y_score_svm = svm.decision_function(x_test_escalonado)
print(f"AUC: {roc_auc_score(y_test, y_score_svm):.4f}")

RocCurveDisplay.from_predictions(y_test, y_score_svm, name='SVM')
plt.plot([0, 1], [0, 1], 'k--', label='Aleatório')
plt.title('Curva ROC - SVM (11 features)')
plt.legend()
plt.show()
```

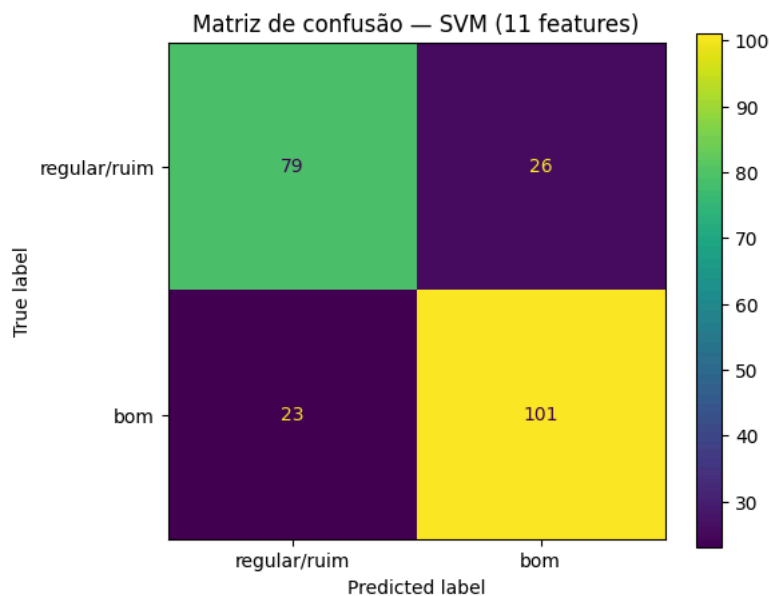


8.2.3 Matriz de Confusão - 11 variáveis

```
matriz_confusao_svm = confusion_matrix(y_true=y_test,
                                       y_pred=y_pred_svm,
                                       labels=[0, 1])

disp = ConfusionMatrixDisplay(confusion_matrix=matriz_confusao_svm,
                              display_labels=['regular/ruim', 'bom'])

fig, ax = plt.subplots(figsize=(6, 5))
disp.plot(values_format='d', ax=ax)
ax.set_title('Matriz de confusão - SVM (11 features)')
plt.show()
```



8.2.4 Criando Modelo com 4 variáveis mais correlacionadas

```
#Melhorando Hiperparametro
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
param_grid_svm_corr = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 0.001, 0.01, 0.1, 1],
    'kernel': ['rbf']
}

grid_svm_corr = GridSearchCV(SVC(random_state=42), param_grid_svm_corr,
                              scoring='accuracy', cv=5, n_jobs=-1, verbose=1)
```

```
grid_svm_corr.fit(x_train_escalado_corr, y_train_corr)
print('SVM:', grid_svm_corr.best_params_)
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits
SVM: {'C': 100, 'gamma': 0.01, 'kernel': 'rbf'}

```
#resultado
svm_corr = grid_svm_corr.best_estimator_
y_pred_svm_corr = svm_corr.predict(x_test_escalado_corr)
print(f"Acurácia: {accuracy_score(y_test_corr, y_pred_svm_corr):.4f}")
```

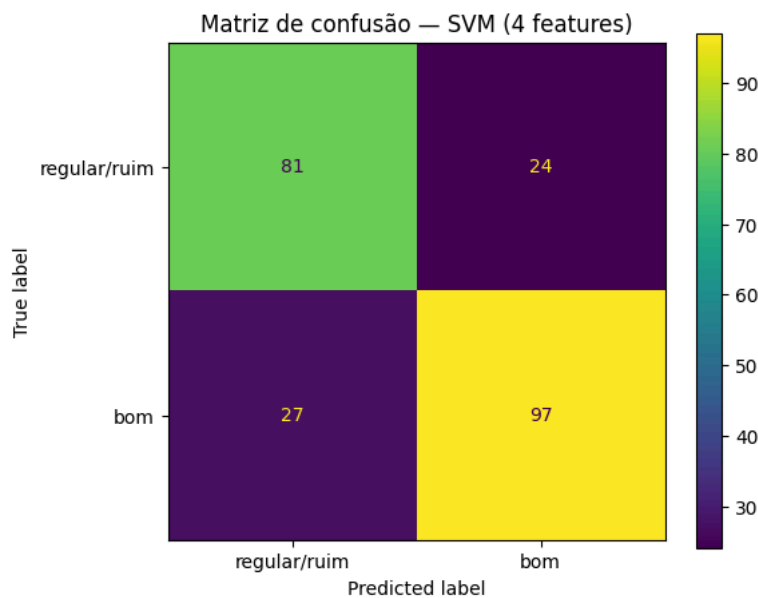
Acurácia: 0.7773

8.2.5 Matriz de Confusão 4 variáveis mais correlacionadas

```
matriz_confusao_svm = confusion_matrix(y_true=y_test_corr,
                                       y_pred=y_pred_svm_corr,
                                       labels=[0, 1])

disp = ConfusionMatrixDisplay(confusion_matrix=matriz_confusao_svm,
                              display_labels=['regular/ruim', 'bom'])

fig, ax = plt.subplots(figsize=(6, 5))
disp.plot(values_format='d', ax=ax)
ax.set_title('Matriz de confusão - SVM (4 features)')
plt.show()
```



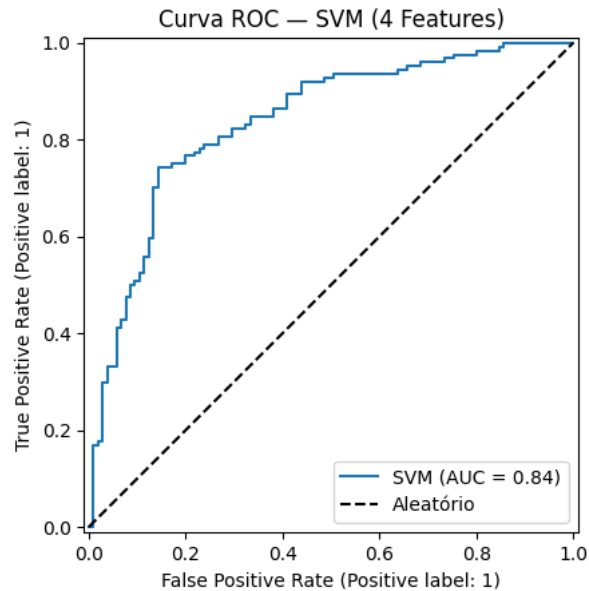
o SVM de 4 features é o mais conservador de todos os modelos até agora — só 24 falsos positivos, o menor número. Ele acerta mais vinhos regulares (81) e menos vinhos bons (97). Enquanto o KNN tende a aprovar demais, esse tende a reprovar

8.2.6 Curva ROC 4 variáveis mais correlacionadas

```
y_score_svm_corr = svm_corr.decision_function(x_test_escalado_corr)
print(f"AUC: {roc_auc_score(y_test_corr, y_score_svm_corr):.4f}")

RocCurveDisplay.from_predictions(y_test_corr, y_score_svm_corr, name='SVM')
plt.plot([0, 1], [0, 1], 'k--', label='Aleatório')
plt.title('Curva ROC - SVM (4 Features)')
plt.legend()
plt.show()
```

AUC: 0.8379



8.2.7 Observações SVM

-- Accuracy

O SVM foi testado com os mesmos dois conjuntos. Com as 11 variáveis, a acurácia foi de 78,6%; com as 4 de maior correlação, 77,7%. Ambos bem acima da baseline de 54,2%.

As matrizes de confusão mostram o comportamento oposto ao do KNN: com 4 variáveis, foram 24 vinhos regulares classificados como bons contra 27 bons classificados como regulares. Ou seja, o modelo é mais restritivo que permissivo.

A diferença de 0,9 ponto entre os conjuntos equivale a 2 vinhos de 229 e não é significativa. Diferente do KNN, o SVM piora levemente ao reduzir para 4 variáveis, pois o kernel RBF já pondera as dimensões internamente.

-- Curva ROC

O AUC foi de 0,842 com 11 variáveis e 0,838 com 4. Ambos bem acima de 0,5, confirmando boa separação das classes.

Como o AUC e acurácia medem coisas diferentes, não devem ser comparadas entre si. Com 4 variáveis mais correlacionadas o modelo acaba performando pior.

Comece a programar ou [gere código](#) com IA.